

用skills固化日常工作

— WPS, a click to unlimited possibilities

汇报人：于智洋



Part One

基础概念

Agent是什么？Skills又是什么？Plugin？

- Agent = LLM + 知识库 + 工具调用 #比如小龙虾、Claude Code等
- Skills = 固定流程 (+ 特定知识库 + 特定工具)

skills 可以通过Agent自动调用或者主动触发

- Plugin = skills + hook + MCP服务 (可以理解为多个skills的载体，还可以同时包含多个hooks和多个MCP服务，也就是功能集合；hook可以理解为自动触发逻辑，比如会话启动时/使用工具前后/收到消息时/定时任务)

日常工作中，通用技能（比如会议纪要、翻译润色）或者一次性流程 没必要建skills，过多的skills会影响agent选择。

MCP服务是什么？

- MCP服务是一个独立的Node.js/Python脚本，负责向Agent暴露一个工具：

1. 通过Npm安装mcp/sdk 协议

2. Server.js 中编写服务代码

* 创建MCP服务实例server，创建实例名，声名该实例能提供工具

* 注册工具列表，告诉claude工具名以及该工具能干什么

* 注册执行逻辑，即一旦被调用具体该如何执行

* 启动服务器，通过标准输入输出格式与Claude通信

3. 在 [plugin-name].mcp.json配置MCP服务

```
import { Server } from '@modelcontextprotocol/sdk/server/index.js';
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js';
import { z } from 'zod';

// 1. 创建服务器实例
const server = new Server({
  name: "calculator-server",
  version: "1.0.0"
}, {
  capabilities: {
    tools: {} // 启用工具能力
  }
});

// 2. 定义并注册工具
server.tool({
  // 工具名称 (全局唯一)
  name: "calculate",
  // 工具描述 (LLM会阅读这个来决定是否调用)
  description: "执行基本的数学运算，支持加减乘除",
  // 输入参数Schema (使用Zod定义)
  inputSchema: z.object({
    operation: z.enum(['add', 'subtract', 'multiply', 'divide']),
    a: z.number().describe('第一个操作数'),
    b: z.number().describe('第二个操作数')
  })
});

// 工具处理函数
async ({ operation, a, b }) => {
  let result;
  switch (operation) {
    case 'add': result = a + b; break;
    case 'subtract': result = a - b; break;
    case 'multiply': result = a * b; break;
    case 'divide':
      if (b === 0) throw new Error('除数不能为零');
      result = a / b;
      break;
  }

  // 3. 返回标准化结果
  return {
    content: [{
      type: 'text',
      text: `计算结果: ${result}`
    }]
  };
};

// 4. 启动服务器
async function main() {
  const transport = new StdioServerTransport();
  await server.connect(transport);
  console.error('计算器MCP服务器已启动');
}

main();
```



Part Two

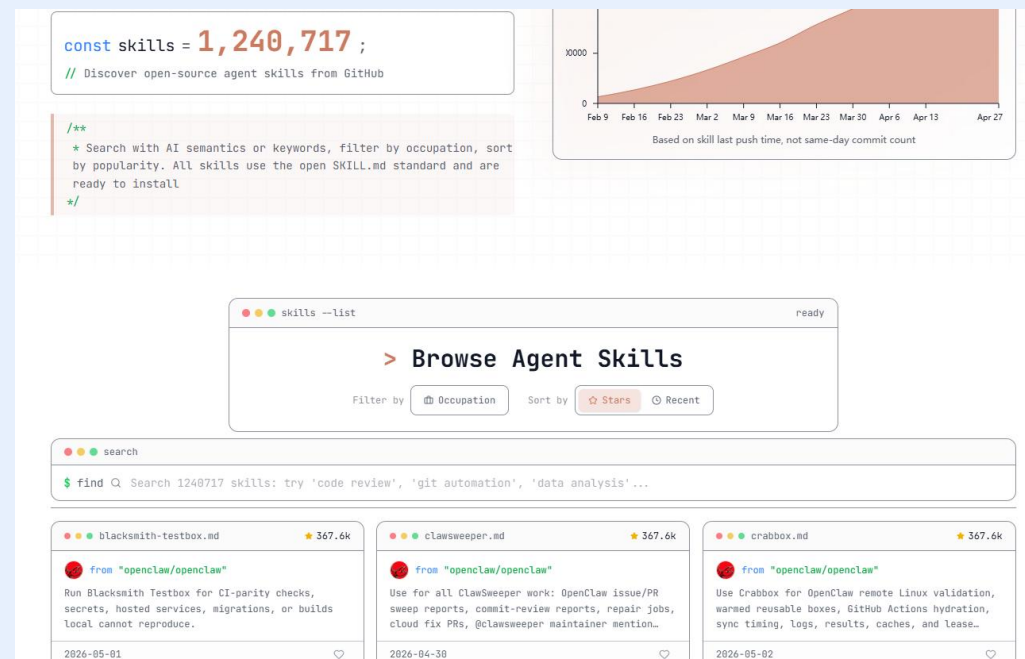
如何安装外部skills

外部skills安装

- 官方预制skills: <https://github.com/anthropics/skills> 主要是各类文档处理及示例 skills (海报设计、前端设计、用playwright测试本地应用、MCP服务创建等)
- 官方plugin: 通过 `/plugin - Marketplaces` 可以添加官方插件市场。比如 `anthropics/skills` (用户名 / 仓库名) 这个 marketplace 中批量安装 `document-skills`
- <https://skillsmp.com/> 或者 `github` 上自己搜, 手动放到 `~/.claude/skills` 下, 或者 `npx skills add skill-name`

```
/plugin
Plugins Discover Installed Marketplaces (</> or tab to cycle)
Install Plugins
> ( ) claude-api
  Claude API and SDK documentation skill for building LLM-p...
( ) document-skills
  Collection of document processing suite including Excel, ...
( ) example-skills
  Collection of example skills demonstrating various capabi...

Space to toggle • Enter to details • Esc to back
```



如何使用skills

- 对话框里直接声名我要用哪个/skill，然后自然交互
- 正确描述要让claude-code做的工作，让claude-code自己判断该用哪个skill

/agent-browser

Thought for 1s >

- 已加载 **agent-browser** 浏览器自动化技能。
这是一个强大的浏览器自动化工具，可以：

- **导航操作** - 打开网页、前进/后退、刷新
- **页面分析** - 获取页面快照、交互元素列表
- **交互操作** - 点击、填写表单、键盘输入、拖拽
- **信息获取** - 获取文本、属性、截图、PDF
- **状态管理** - 保存/加载登录状态
- **网络拦截** - 模拟响应、阻止请求
- **移动端模拟** - iPhone、iPad 等设备模拟

基本工作流程：

1. `agent-browser open <url>` - 导航到页面
2. `agent-browser snapshot -i` - 获取交互元素（返回 @e1, @e2 等引用）
3. `agent-browser click @e1` - 使用引用进行操作

您想用浏览器自动化做什么？例如：

- 测试某个网站
- 填写表单
- 截取网页截图
- 抓取网页数据
- 其他自动化任务

</> c:\Users\Zhiyang\.claude\ski...

打开www.ouyeel.com，登陆我的账号，查看下我的加车里有没有降价资源

- 我来帮你打开欧冶云商网站并登录查看加车里的降价资源。这需要使用浏览器自动化功能。
- **web-access** skill
- 我来帮你完成这个任务。首先检查 CDP 模式可用性。
- **Bash** 检查 CDP 模式依赖

```
IN node "C:/Users/Zhiyang/.claude/skills/web-access/scripts/check-deps.mjs"
```

* Calculating...



Part Three

如何构建自己的skills

skills文件结构

技能命名：〈团队〉-〈功能〉-〈描述词〉 比如：营销团队-终端识别-行业分类/地理位置解析

Skills文件结构：

SKILL.md	核心流程
scripts/	可执行脚本文件夹
references/	参考文件（特定知识库）
assets/ 或者 templates/	模板（比如输出模板）
examples/	示例

* 以上目录、目录命名并非强制，只有 SKILL.md是必须的

SKILL.md 文件构成 (1)

- 1. 用 `---` / `---` 包裹的 YAML 数据: `name + description(+allowed-tools + user-invocable + context)` 负责定义该skills的功能以及调用时机, 实现【目录】的效果, 智能体在初始化时加载所有可用skills的该目录, 就能知道何时应该调用哪个skill;
- 有严格格式要求: 关键字段如冒号后必须有一个空格、name描述清晰 (英文动名词 + 连字符-);
- `allowed-tools` 让skill调用被声明的tools时无需人工审核, 其它工具不能用;
- `user-invocable` 设置为 `false`时该skill无法被 / 命令菜单展示;
- 注意: `description`中为了适配用户可能存在的多样化表达, 需要枚举关键词, 构建一个“意图-多个近似关键词”映射

SKILL.md 文件构成 (2)

- 2. Markdown 指令主体，即定义具体流程，包括特定知识库查询及特定工具使用。
- # [skill-name]
- ## Quick Start -- 负责告诉用户如何使用该skill，以及让大模型更精准理解该skill的功能
- ## Instruction -- 有序执行列表（核心），重要步骤要用核对表checklist形式：- [] 第N步：xxx （表示未完成，做完后claude会验证并勾掉）；- [x] kkkkk （表示已完成）让Agent慢思考并展示进度详情；
- 可以定义决策树，比如根据用户上传文件后缀不同： * 若文件为csv，则执行a.py * 若文件为 xlsx，则执行b.py
- 脚本输出必须打印出结构化json字符串，让智能体能理解结果并做出下一步决策。可以直接在SKILL.md文件中给出返回示例：“脚本执行成功后会返回如下格式的json，请从中提取aaaa字段和bbbb字段并向用户报告”
- ## Examples -- 给定2~3组输入输出示例即可，也可包括异常处理回复范式
- 可以同时给出反面示例以划定红线：“回复应避免以下风格 * A回复（A回复为何不好） * B回复（B回复为何不好）”
- ## Resources -- 引用 references/xxx.md 参考文件进行调用

SKILL.md 文件构成（注意事项）

* 保证SKILL.md文件行数不要太多，保持在500行内为佳，多余内容如详细规则等可以放在reference 文件夹内引用

* 涉及到脚本执行时，必须给出完整的命令行示例：“`bash python scripts/abc.py --para1 <字符串> --para2 <列表>`” 参数详解： * `--para1`:（必填，字符串）企业名称 * `--para2`:（必填，列表）英文逗号分隔，可选值包括：... ..”

* 异常处理：

【不留痕迹】若结果生成失败，要规定skill捕获异常并删除中间文件

【不破坏环境】关键文件（如配置文件）修改，要让skill遵循 备份-修改-验证-失败回滚 的操作

【二次确认】高风险操作时（如改动数据库或转账等），必须让skill在执行前要求用户二次确认：停止操作 - 向用户反馈警告，提示用户请回复确认执行 - 规定仅当用户明确确认执行时才执行

skills中怎么调用MCP tools

MCP服务（被调用方）：

声明三要素，TypeScript或python都行：

名称：大模型调用唯一标识

描述：帮助模型理解工具能力（声名此功能用于差旅预定，模型根据用户指令匹配该工具）

输入架构：定义入参出参的数据类型和含义（模型生成符合Schema的参数请求，通过MCP发送，在沙箱里执行并返回结果）

* 结果数据会返回模型上下文进行确认及迭代

【安装】`claude mcp add <服务名称> -s <命名空间> --<启动命令>`

`-s/--scope`：命名空间（`user`=个人，`global`=全局）

【自建】把MCP脚本文件地址配置到 Claude Code 的 `mcp.json`

SKILL.md（调用方）：

声明：在SKILL文件的YAML头部，明确列出该技能被授权调用的MCP工具白名单

```
allowed-tools: [ "mcp_plugin_<plugin-name>_<server-name>_<tool-name>",  
  "mcp_plugin_filesystem_filesystem_read_file" ]
```

按流程触发调用该 mcp工具 ‘MCP服务实例名/工具名’，用正常的自然语言写就行，Claude-code会根据实际需要自主调用MCP-tools

skills中怎么执行Python或Bash脚本

Python脚本（被调用方）：

要用 `argparse`库来接收参数：

```
import argparse
```

```
import sys
```

```
parser = argparse.ArgumentParser() #定义参数接收器
```

```
parser.add_argument("--para1", required=True)
```

```
parser.add_argument("--para2", required=True)
```

```
args = parser.parse_args() #解析参数，后续代码中  
用 args.para1 和 args.para2 来调用
```

【输入】注意脚本调用入参时也要做一道前置校验，防止入参格式有误，若有错误，除了脚本执行要停掉，还要将ERROR及原因打印出来，这样claude-code看到报错会自动修正入参并重试。

【输出】claude-code只能理解脚本打印出来的内容，最好的方式是将JSON格式的数据打印出来

SKILL.md（调用方）：

开启沙箱，为脚本提供独立运行环境：文件隔离（比如删除中间文件只能删tmp目录下的）、网络访问控制（只能请求内网API）、
`autoAllowBashIfSandboxed: true` 配置让安全边界内的操作可以省略人工审批

要想执行python脚本，skills.md中可以指定用bash的方式：`python scripts/abc.py --para1 aaa --para2 200`

注意在从用户提示词中识别aaa、200等入参时，必须要明确数据格式，防止模型自由发挥

文件中要规定：脚本输出json对象，但不要向用户展示原始json内容；读取 `status` 字段，如果该字段为false，向用户发出警告；用自然语言总结json结果

skills中怎么查询数据

- 【SQL】要将常用查询封装成脚本函数，只暴露查询条件，不要让大模型自己生成SQL：

Scripts/order_query.py

```
sql = "select * from dm.t_dd_orders where cmpy_id = %s"
```

```
cursor.execute(sql, (cmpy_id,))
```

对应的skill.md脚本中明确执行 `python scripts/order_query.py --cmpy_id T10773`

不允许大数据量返回，尽量库内计算

- 【外挂知识库】只召回关键信息（类似dify知识库的分段召回，父子、前后分段，可以用简单脚本也可以借助dify做个RAG）
- 【外部资源】渐进式披露 -- 符合某个条件时再索引到对应目录下的某个文件



Part Four

看看别人skills怎么写的

Agent-browser skill

- 自动操作浏览器的命令行工具，可以实现表单填写、网页截图、抓取数据等功能：
- 元数据定义：名称、用途及调用场景
- 快速开始：简单指令示例，比如打开页面、点击、填充等简单操作
- 核心 workflow Core workflow:

【导航到目标网页-> 扫描网页并获取页面上可操作元素（比如输入框、按钮、链接）列出来，并给每个元素编个号 ref = e# -> 根据上述编号进行元素操作：点击/填写/键盘按键 -> 等待浏览器跳转到指定页面wait --url或者等所有网络请求结束 wait --load DOM变化后，重新快照获取新元素 -> 重复以上步骤直到结束】

- 核心命令集 Commands：将各类能力分配到不同的模块下。比如我写“打开百度页面，搜索...”，claude-code会先从内部工具仓库或云端拉取 agent-browser工具的代码，然后根据我的提示词映射到 agent-browser open www.baidu.com 命令，然后在claude-code自带的runtime环境里启动 agent-browser 并执行
- 典型场景示例：表单提交、保存登陆状态（不用重复登陆）、基于Header认证直接跳过登录流程、账号密码加密保存本地而非明文保留在命令历史中
- 会话与持久化配置：会话及cookie保存
- 基础补充：明确用bun包管理器安装agent-browser包；支持 -- help命令参数自查
全量命令，无需仅搜索SKILL文档

```
C:\> Users > Zhiyang > .claude > skills > agent-browser > SKILLmd > ** # Browser Automation with agent-browser > ** ## Native Mode (Experimental)
1 ---
2 name: agent-browser
3 description: Automates browser interactions for web testing, form filling, screenshots, and data extraction. Use when the user needs to
  navigate websites, interact with web pages, fill forms, take screenshots, test web applications, or extract information from web pages.
4 ---
5
6 # Browser Automation with agent-browser
7
8 ## Quick start
9
10 ```bash
11 agent-browser open <url>      # Navigate to page
12 agent-browser snapshot -i     # Get interactive elements with refs
13 agent-browser click @e1       # Click element by ref
14 agent-browser fill @e2 "text" # Fill input by ref
15 agent-browser close          # Close browser
16 ```
17
18 ## Core workflow
19
20 1. Navigate: `agent-browser open <url>`
21 2. Snapshot: `agent-browser snapshot -i` (returns elements with refs like `@e1`, `@e2`)
22 3. Interact using refs from the snapshot
23 4. Re-snapshot after navigation or significant DOM changes
24
25 ## Commands
26
27 ### Navigation
28 ```bash
29 agent-browser open <url>      # Navigate to URL (aliases: goto, navigate)
30 agent-browser back           # Go back
31 agent-browser forward        # Go forward
32 agent-browser reload         # Reload page
33 agent-browser close          # Close browser (aliases: quit, exit)
34 ```
35
36 ### Snapshot (page analysis)
37 ```bash
38 agent-browser snapshot        # Full accessibility tree
39 agent-browser snapshot -i     # Interactive elements only (recommended)
40 agent-browser snapshot -i -C  # Include cursor-interactive elements (divs with onclick, etc.)
41 agent-browser snapshot -c     # Compact (remove empty structural elements)
42 agent-browser snapshot -d 3   # Limit depth to 3
43 agent-browser snapshot -s "#main" # Scope to CSS selector
44 agent-browser snapshot -i -c -d 5 # Combine options
45 ```
46
47 The "-C" flag is useful for modern web apps that use custom clickable elements (divs, spans) instead of standard buttons/links.
48
49 ### Interactions (use @refs from snapshot)
50 ```bash
51 agent-browser click @e1       # Click (--new-tab to open in new tab)
52 agent-browser dblclick @e1    # Double-click
53 agent-browser focus @e1       # Focus element
54 agent-browser fill @e2 "text" # Clear and type
55 agent-browser type @e2 "text" # Type without clearing
56 agent-browser keyboard type "text" # Type with real keystrokes (no selector, current focus)
57 agent-browser keyboard inserttext "text" # Insert text without key events (no selector)
58 agent-browser press Enter     # Press key
59 agent-browser press Control+a # Key combination
60 agent-browser keydown Shift   # Hold key down
61 agent-browser keyup Shift     # Release key
62 agent-browser hover @e1       # Hover
63 agent-browser check @e1       # Check checkbox
64 agent-browser uncheck @e1     # Uncheck checkbox
65 agent-browser select @e1 "value" # Select dropdown
66 agent-browser scroll down 500  # Scroll page (--selector <sel> for container)
```

```
---
Install: `bun add -g agent-browser && agent-browser install`. Run `agent-browser --help` for all commands. R
agent-browser
```

Agent-browser 衍生出的新skills 赋能日常工作

- <https://github.com/heimanba/order-analysis-skill> 工单分析skill
- 执行 `scripts/check-cdp.sh` 校验 Chrome Debug模式和 `agent-browser`工具是否就绪
- 用 `agent-browser`打开jira页面
- 按时间戳创建输出目录
- 执行JS脚本，获取工单数据并写入文件
- 大模型分析工单数据



大部分面向电脑屏幕的工作都可以被固化为Skills